



Cinvestav



COCYTEN
CONSEJO DE CIENCIA Y TECNOLOGÍA
DEL ESTADO DE NAYARIT

LAB 8 Shift Register

Student: Alonso Emmanuel López Macías

Email's: usuario_cocyten_32_x@cinvestav.mx
alemlopezma@gmail.com

Student: Cesar Eduardo Inda Cenicerros

Email's: usuario_cocyten_32_x@cinvestav.mx
cesar.inda@iteso.mx

Full Professor: Dr. Martin Gonzalez Perez

23/04/2025

CINVESTAV, Av Del Bosque 1145, El Bajío 45017, Zapopan Jalisco.
COCYTEN, Boulevard Luis Donaldo Colosio. Ciudad Industrial, 63173 Tepic Nayarit.
Technical Report

® **CINVESTAV - COCYTEN**

Abstract: *FPGA Terasic
Board – ALTERA Cyclone IV EP4CE115F29C7N
IDE – Quartus Prime 18.1
Boolean Algebra
Digital Gates & Logical Comparators
Git & GitHub Control Version*

Keywords: *Shift Register, Sequential Logic, Parallel, Delays, Asynchronous, Rotation & Reset Signal.*

Table of Contents

Table of Contents.....	ii
Table of Figures	iii
1.- INTRODUCTION	0
1.1.- Requirements.....	0
2.- FUNCTIONAL DESCRIPTION	1
2.1.- Boolean Algebra Equations.....	2
2.1.- TrueTable & Results.....	3
2.2.- GitHub Repository	4
3.- ISSUES	5
4.- CONCLUSIONS.....	5

Table of Figures

Figure: 1 Shift Register Flip-Flops.....	0
Figure: 2 ShiftRegister_bdf.....	1
Figure: 3 D-Flip_Flop.....	1
Figure: 4 PinPlanner.....	2
Figure: 5 RTL_Viewver.....	2
Figure: 6 TrueTable_ShiftRegister.....	3
Figure: 7 Test1.....	4
Figure: 8 Test2.....	4
Figure: 9 Test3.....	4
Figure: 10 GitHub Repository.....	4

1.- INTRODUCTION

The operation of a shift register is fundamental because it represents one of the basic structures for sequential data manipulation. A shift register allows storing and shifting bits of information in response to clock signals, making it crucial for tasks such as data serialization and deserialization, controlled delays, digital filtering, and I/O port expansion. Flip-flops and LUTs are configured to implement shift registers, creating highly parallel, resource-optimized, and high-speed processing solutions. Having a clear understanding of shift registers not only facilitates the implementation of basic functions like counters, data buffers, or serial communication systems, but also provides the foundation for tackling more complex architectures such as state machines, digital filters, and signal processors within an FPGA. The shift register is an essential tool in the arsenal of any digital designer aiming to fully harness the potential of FPGAs in high-efficiency, high-reliability projects. For this reason, in this laboratory, we were tasked with implementing a shift register, highlighting its importance while also gaining a clear understanding of how sequential circuits begin to operate within each of these implementations.

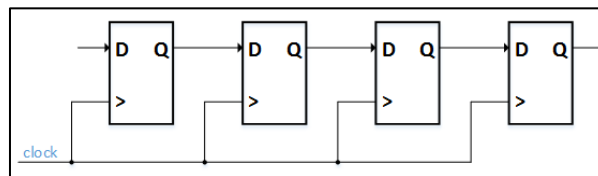


Figure: 1 Shift Register Flip-Flops

1.1.- Requirements

Design a shift register with the following characteristics:

- 1.- The shift register must be **8 bits wide**.
- 2.- The circuit must include an **asynchronous reset signal (arstn)**.
- 3.- The register must have an **input (in)** to allow data to be introduced into the shift register.
- 4.- The circuit must include an **enable input (shift_en)** to control the bit shifting.
- 5.- The circuit must include a **direction input (dir)**:
 - If $dir = 1$, the shift must be to the left.
 - If $dir = 0$, the shift must be to the right.
- 6.- The circuit must include a **rotation input (rot_in)** that enables data insertion:
 - If $rot_in = 1$, the value of in is inserted into the shift register at the **MSb or LSB**, depending on the **dir** value.
 - If $rot_in = 0$, the bits in the register rotate:
 - The **MSb moves to the LSB** when $dir = 1$.
 - The **LSb moves to the MSb** when $dir = 0$.

2.- FUNCTIONAL DESCRIPTION

The project was created using key library/block elements such as **"mux"**, **"busmux"**, and **"dffe"**. For each of these elements, a concatenation was made on their respective inputs. For example, to specifically use the D-type flip-flop, an adjustment was made with the bus multiplexer to simplify the use of flip-flops, resulting in a general circuit defined in this manner.

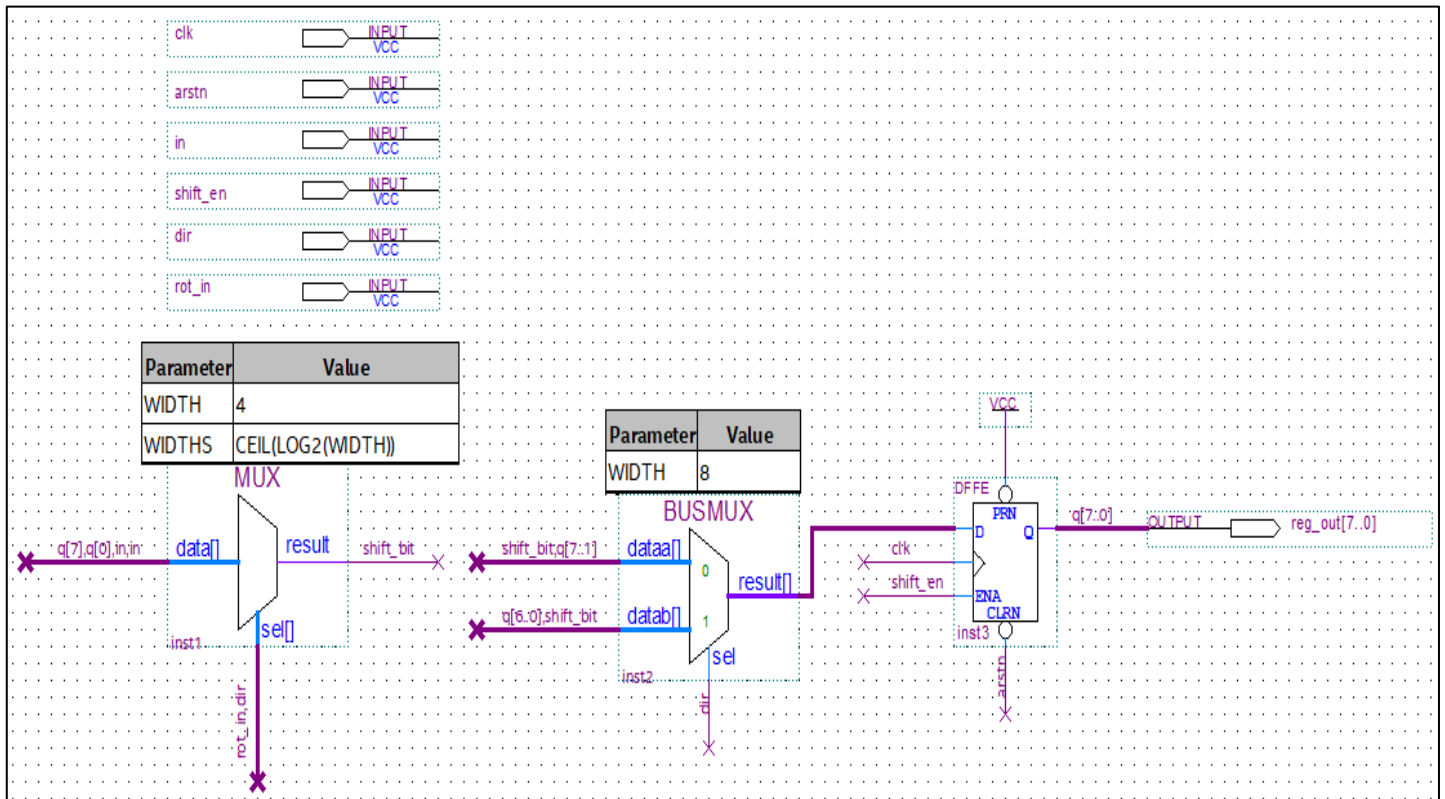


Figure: 2 ShiftRegister_bdf

The next element is a simplification:

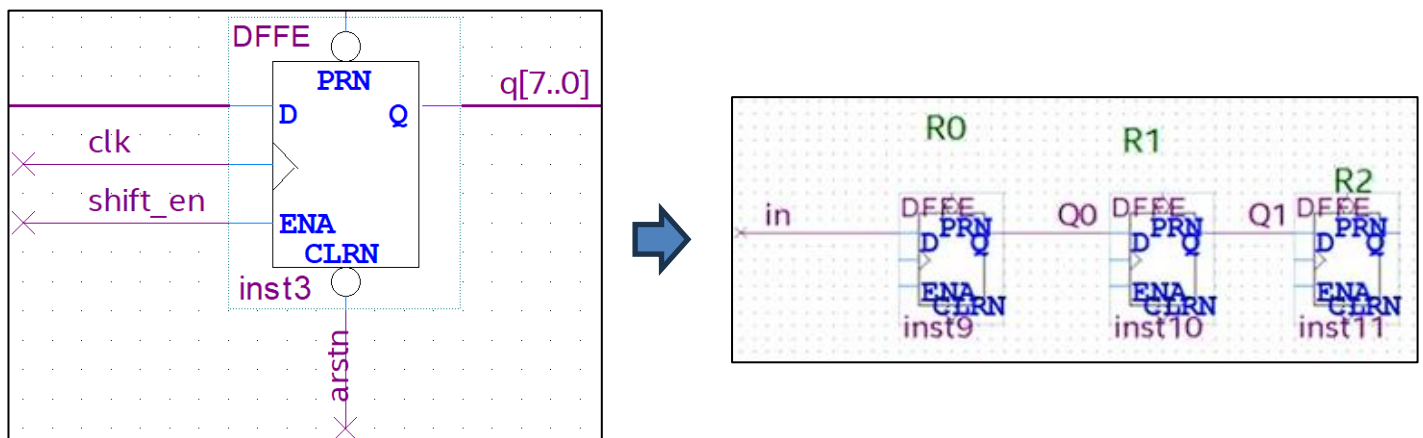


Figure: 3 D-Flip_Flop

The pin assignment in the “Pin Planner” was as follows:

Node Name	Direction	Location
in arstn	Input	PIN_M21
in clk	Input	PIN_M23
in dir	Input	PIN_Y24
in in	Input	PIN_AB28
out reg_out[7]	Output	PIN_G21
out reg_out[6]	Output	PIN_G22
out reg_out[5]	Output	PIN_G20
out reg_out[4]	Output	PIN_H21
out reg_out[3]	Output	PIN_E24
out reg_out[2]	Output	PIN_E25
out reg_out[1]	Output	PIN_E22
out reg_out[0]	Output	PIN_E21
in rot_in	Input	PIN_AA22
in shift_en	Input	PIN_Y23

Figure: 4 PinPlanner

RTL VIEWER

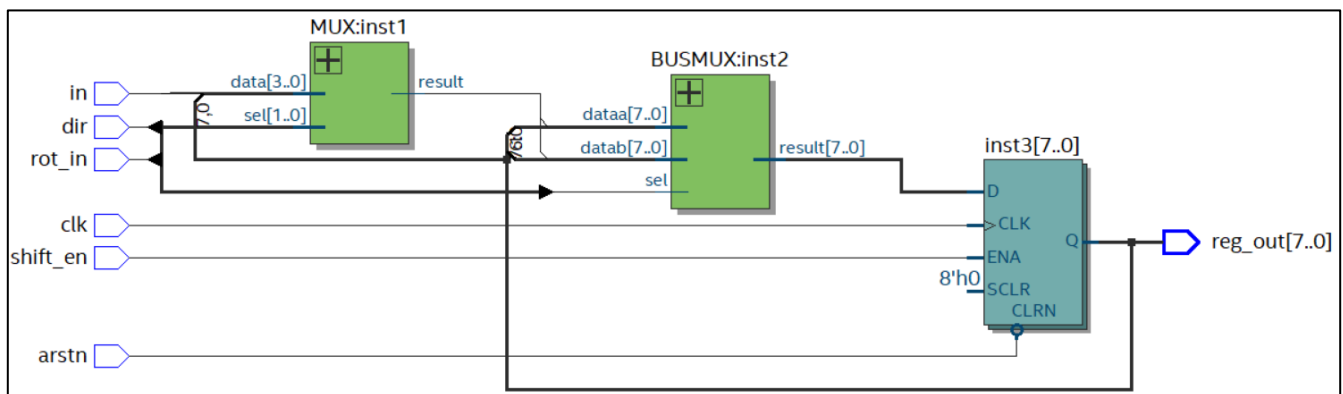


Figure: 5 RTL_Viewver

2.1.- Boolean Algebra Equations

The general expression is:

$$S = (rot_{in} \wedge in) \vee \sim rot_{in} \wedge dir \wedge Q7) \vee (\sim rot_{in} \wedge \sim dir \wedge Q0)$$

In this case rot_in AND set in if “rot_in” is activate.

If don’t exist a rotation and the shift is to left, we use a “Q7”.

If don’t exist a rotation and the shift is to right, we use a “Q0”.

The equations for each one state can be:

$$\begin{aligned}
 next_Q[7] &= (\sim dir \wedge S) \vee (dir \wedge Q[6]) \\
 next_Q[6] &= (\sim dir \wedge Q[7]) \vee (dir \wedge Q[5]) \\
 next_Q[5] &= (\sim dir \wedge Q[6]) \vee (dir \wedge Q[4]) \\
 next_Q[4] &= (\sim dir \wedge Q[5]) \vee (dir \wedge Q[3]) \\
 next_Q[3] &= (\sim dir \wedge Q[4]) \vee (dir \wedge Q[2]) \\
 next_Q[2] &= (\sim dir \wedge Q[3]) \vee (dir \wedge Q[1]) \\
 next_Q[1] &= (\sim dir \wedge Q[2]) \vee (dir \wedge Q[0]) \\
 next_Q[0] &= (\sim dir \wedge Q[1]) \vee (dir \wedge S)
 \end{aligned}$$

2.1.- TrueTable & Results

The truth table for this circuit would be as follows

<i>dir</i>	<i>rot_in</i>	<i>in</i>	<i>Q7</i>	<i>Q0</i>	<i>S</i>
0	0	0	0	0	0
0	0	0	0	1	1
0	0	0	1	0	0
0	0	0	1	1	1
0	0	1	0	0	0
0	0	1	0	1	1
0	0	1	1	0	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	0	0	1	0
0	1	0	1	0	0
0	1	0	1	1	0
0	1	1	0	0	1
0	1	1	0	1	1
0	1	1	1	0	1
0	1	1	1	1	1
1	0	0	0	0	0
1	0	0	0	1	0
1	0	0	1	0	1
1	0	0	1	1	1
1	0	1	0	0	0
1	0	1	0	1	0
1	0	1	1	0	1
1	0	1	1	1	1
1	1	0	0	0	0
1	1	0	0	1	0
1	1	0	1	0	0
1	1	0	1	1	0
1	1	1	0	0	1
1	1	1	0	1	1

Figure: 6 TrueTable_ShiftRegister

To test the circuit's behavior, both the data and the corresponding "enable" (SW17) were initially set high (logic 1 or true), along with "IN" (SW0). Clock pulses were then applied to increase the number of bits stored in the flip-flop.



Figure: 7 Test1

ENABLE = 1
IN = 1
CLOCK = 1 TIME



Figure: 8 Test2

ENABLE = 1
IN = 1
CLOCK = 8 TIME



Figure: 9 Test3

ENABLE = 1
IN = 1
RESET = 1

2.2.- GitHub Repository

A branch was created in the course repository corresponding to Lab8_ShiftRegister.

https://github.com/CeicGitHub/Fundamentals_Of_Digital_Design_FPGA_1/tree/QuartusLab8_ShiftRegister

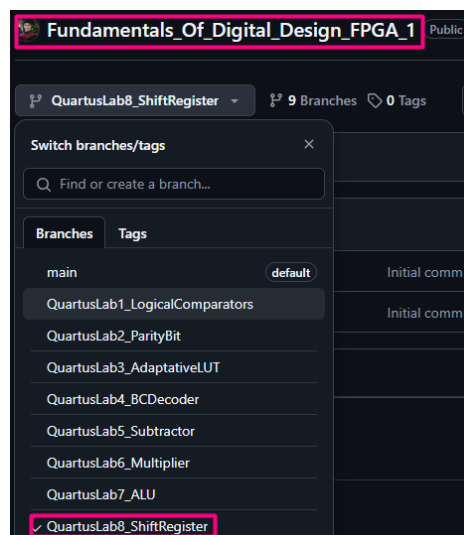


Figure: 10 GitHub Repository

3.- ISSUES

Alonso Emmanuel López Macías

The shift from combinational to sequential logic represented a major change, as it required considering a lot of varying logic throughout the course. In this particular case, implementing a D-type flip-flop was somewhat confusing to understand at first. However, once the lab was conducted, the operation on the FPGA became a bit clearer. I believe one of the challenges was that, although we were assisted during the implementation, it made it harder for us to fully observe and understand some of these details during testing.

Cesar Eduardo Inda Cenicerros

Personally, I don't often work with elements as abstract as flip-flops or digital system design. Although I implicitly use them daily when programming in C, seeing them at this lower level requires a deeper understanding of logic and the behavior of individual elements and blocks. Specifically, understanding why, when the IN-value changes to 0, the data shifts and fills with zeros to the left or right, was a control state related to shifting, something we usually program with operators like << or >>. In this case, however, it was a challenge to learn and revisit this kind of logic.

4.- CONCLUSIONS

Alonso Emmanuel López Macías

Through this lab, I learned a lot. It allowed me to recall and reinforce my knowledge of sequential logic and to better understand the importance of elements like flip-flops. I know that in this particular case, a logical shift was implemented, and this also helped me understand the impact it has on memory not just as storage, but in how these operations act on the stored data to generate many actions.

Cesar Eduardo Inda Cenicerros

I learned a lot about the operation and importance of flip-flops in sequential digital systems and the impact they have, especially in shifter processes. I realized how critical it can be if a shift occurs toward another section of memory or elements transmitting data. I became aware of the importance of clearly understanding how these electronic components work and how it is possible to design them starting from a truth table.